```
# Install required packages
import subprocess
import sys

packages = ['tensorflow', 'keras', 'pandas', 'numpy', 'plotly', 'scikit-learn']
for package in packages:
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q', package])

print(" All packages installed successfully!")
```

```
 All packages installed successfully!
```

```
# Import libraries
import numpy as np
import pandas as pd
from datetime import datetime, timedelta
import warnings

# TensorFlow & Keras
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Visualization
import plotly.graph_objects as go
import plotly.express as px
from plotly.subplots import make_subplots

# ML utilities
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

warnings.filterwarnings('ignore')

print(" Imports successful!")
print(f"TensorFlow version: {tf.__version__}")
print(f"Pandas version: {pd.__version__}")
```

```
 Imports successful!
TensorFlow version: 2.19.0
Pandas version: 2.2.2
```

```
def generate_electricity_data(n_days=365):
    """
    Generate synthetic hourly electricity consumption data.

    Args:
        n_days: Number of days to generate

    Returns:
        DataFrame with consumption data
    """
    np.random.seed(42)

    # Create hourly timestamps
    start_date = datetime(2023, 1, 1)
    hours = 24 * n_days
    dates = [start_date + timedelta(hours=i) for i in range(hours)]

    data = []

    for date in dates:
        hour = date.hour
        day_of_week = date.weekday()
        month = date.month

        # Determine day type
        if day_of_week >= 5:  # Saturday = 5, Sunday = 6
            day_type = "Weekend"
        elif date.day in [1, 25, 26] or (month == 1 and date.day in range(1, 4)):
            day_type = "Holiday"
        else:
            day_type = "Weekday"

        # Base load depends on hour of day (sinusoidal pattern)
        base_load = 50 + 30 * np.sin(2 * np.pi * (hour - 6) / 24)

        # Adjust for day type
        if day_type == "Weekend":
```

```
            base_load *= 0.85
        elif day_type == "Holiday":
            base_load *= 0.75

        # Peak hours (9-12, 18-21): 30% boost
        if 9 <= hour <= 12 or 18 <= hour <= 21:
            base_load *= 1.3

        # Random noise
        noise = np.random.normal(0, 3)
        consumption = max(10, base_load + noise)

        data.append({
            'date': date,
            'hour': hour,
            'day_of_week': day_of_week,
            'day_type': day_type,
            'month': month,
            'consumption': consumption
        })

    df = pd.DataFrame(data)
    return df

# Generate data
print("Generating electricity consumption data for 1 year...")
df = generate_electricity_data(n_days=365)

print(f"\n Generated {len(df):,} hourly records")
print(f"Date range: {df['date'].min()} to {df['date'].max()}")
print(f"\nDay type distribution:")
print(df['day_type'].value_counts())
```

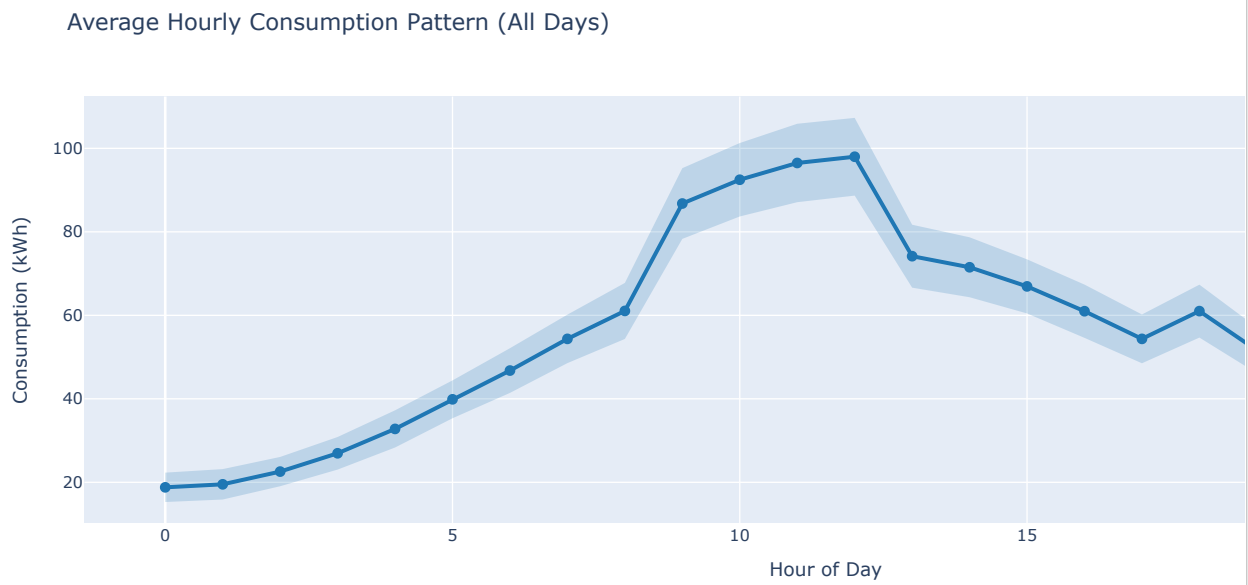
```
Generating electricity consumption data for 1 year...

 Generated 8,760 hourly records
Date range: 2023-01-01 00:00:00 to 2023-12-31 23:00:00

Day type distribution:
day_type
Weekday    5616
Weekend    2520
Holiday     624
Name: count, dtype: int64
```

```
# Display sample data
print("Sample Data (first 10 records):")
print(df.head(10))

print("\nConsumption Statistics:")
print(df['consumption'].describe())

print("\nData Info:")
df.info()
```

```
Sample Data (first 10 records):
                 date  hour  day_of_week day_type  month  consumption
0 2023-01-01 00:00:00     0            6  Weekend      1    18.490142
1 2023-01-01 01:00:00     1            6  Weekend      1    17.454099
2 2023-01-01 02:00:00     2            6  Weekend      1    22.359418
3 2023-01-01 03:00:00     3            6  Weekend      1    29.037867
4 2023-01-01 04:00:00     4            6  Weekend      1    29.047540
5 2023-01-01 05:00:00     5            6  Weekend      1    35.197703
6 2023-01-01 06:00:00     6            6  Weekend      1    47.237638
7 2023-01-01 07:00:00     7            6  Weekend      1    51.402190
8 2023-01-01 08:00:00     8            6  Weekend      1    53.841577
9 2023-01-01 09:00:00     9            6  Weekend      1    80.318270

Consumption Statistics:
count    8760.000000
mean       52.376467
std        25.540171
min        10.000000
25%        29.238925
50%        50.967478
75%        68.961071
max       113.294898
Name: consumption, dtype: float64

Data Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8760 entries, 0 to 8759
Data columns (total 6 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
```

```
 0   date         8760 non-null   datetime64[ns]
 1   hour         8760 non-null   int64
 2   day_of_week  8760 non-null   int64
 3   day_type     8760 non-null   object
 4   month        8760 non-null   int64
 5   consumption  8760 non-null   float64
dtypes: datetime64[ns](1), float64(1), int64(3), object(1)
memory usage: 410.8+ KB
```

```
# Hourly consumption pattern
hourly_pattern = df.groupby('hour')['consumption'].agg(['mean', 'std'])

fig_hourly = go.Figure()

fig_hourly.add_trace(go.Scatter(
    x=hourly_pattern.index,
    y=hourly_pattern['mean'],
    mode='lines+markers',
    name='Average',
    line=dict(color='#1f77b4', width=3),
    marker=dict(size=8)
))

# Confidence bands
fig_hourly.add_trace(go.Scatter(
    x=hourly_pattern.index.tolist() + hourly_pattern.index.tolist()[::-1],
    y=(hourly_pattern['mean'] + hourly_pattern['std']).tolist() +
      (hourly_pattern['mean'] - hourly_pattern['std']).tolist()[::-1],
    fill='toself',
    fillcolor='rgba(31, 119, 180, 0.2)',
    line=dict(color='rgba(255,255,255,0)'),
    name='±1 Std Dev'
))

fig_hourly.update_layout(
    title='Average Hourly Consumption Pattern (All Days)',
    xaxis_title='Hour of Day',
    yaxis_title='Consumption (kWh)',
    hovermode='x unified',
    height=500
)

fig_hourly.show()
print("\n📊 Peak consumption hours: 9-12 (morning) and 18-21 (evening)")
```

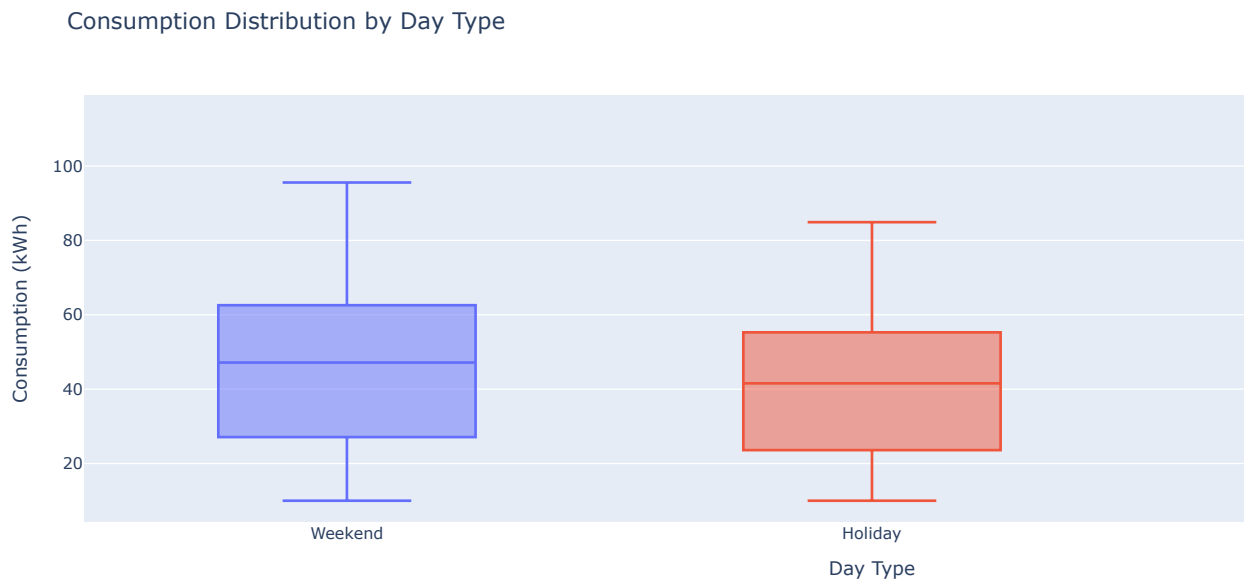



```
📊 Peak consumption hours: 9-12 (morning) and 18-21 (evening)
```

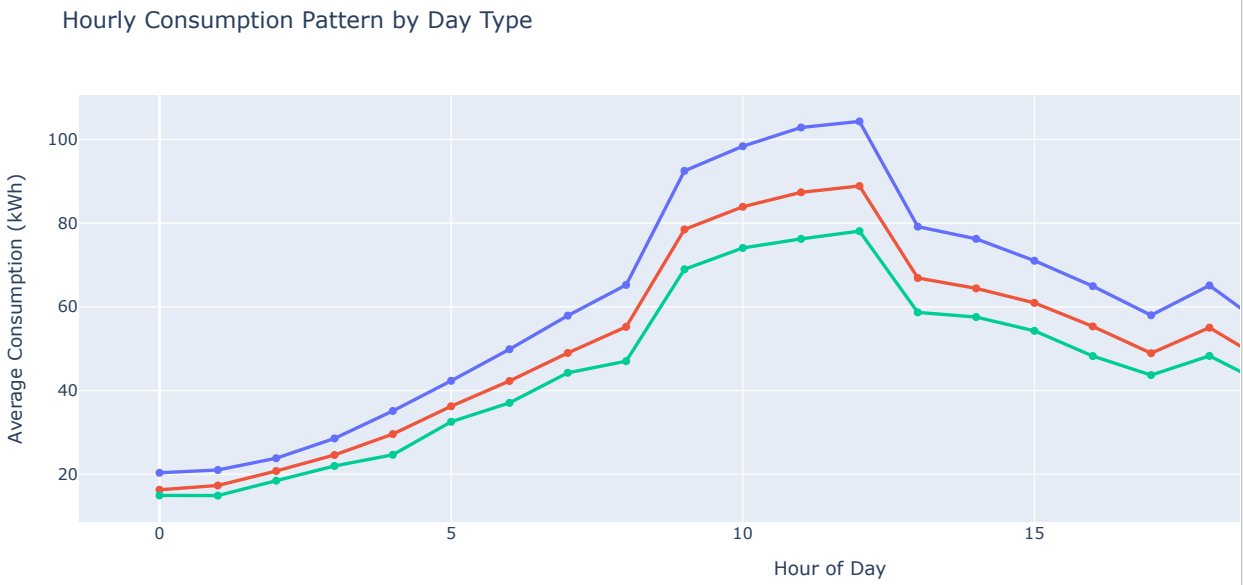
```
# Day type comparison
day_type_stats = df.groupby('day_type')['consumption'].agg(['mean', 'min', 'max', 'std', 'count']).round(2)

print("Consumption Statistics by Day Type:")
print(day_type_stats)

fig_day_type = px.box(
    df,
```

```
    x='day_type',
    y='consumption',
    title='Consumption Distribution by Day Type',
    labels={'day_type': 'Day Type', 'consumption': 'Consumption (kWh)'},
    color='day_type'
)

fig_day_type.update_layout(height=500, showlegend=False)
fig_day_type.show()
```

```
Consumption Statistics by Day Type:
           mean    min     max    std  count
day_type
Holiday   41.69  10.00   84.90  19.99    624
Weekday   55.80  13.36  113.29  26.59   5616
Weekend   47.40  10.00   95.57  22.68   2520
```




```
# Consumption by hour and day type
hourly_by_type = df.groupby(['hour', 'day_type'])['consumption'].mean().reset_index()

fig_comparison = go.Figure()

for day_type in ['Weekday', 'Weekend', 'Holiday']:
    data_type = hourly_by_type[hourly_by_type['day_type'] == day_type]
    fig_comparison.add_trace(go.Scatter(
        x=data_type['hour'],
        y=data_type['consumption'],
        mode='lines+markers',
        name=day_type,
        line=dict(width=2.5)
    ))

fig_comparison.update_layout(
    title='Hourly Consumption Pattern by Day Type',
    xaxis_title='Hour of Day',
    yaxis_title='Average Consumption (kWh)',
    hovermode='x unified',
    height=500
)

fig_comparison.show()
```




```
class LSTMElectricityPredictor:
    def __init__(self, lookback=24, forecast_horizon=6):
        """
        Initialize LSTM model for electricity prediction.

        Args:
            lookback: Number of previous hours to look back
            forecast_horizon: Number of hours to predict ahead
        """
        self.lookback = lookback
        self.forecast_horizon = forecast_horizon
        self.model = None
        self.scaler = MinMaxScaler(feature_range=(0, 1))
        self.history = None
        self.test_metrics = {}

    def create_sequences(self, data):
        """Create sequences for LSTM training/prediction"""
        X, y = [], []
        for i in range(len(data) - self.lookback - self.forecast_horizon + 1):
            X.append(data[i:i + self.lookback])
            y.append(data[i + self.lookback + self.forecast_horizon - 1])
        return np.array(X), np.array(y)

    def build_model(self, input_shape):
        """Build LSTM neural network"""
        model = keras.Sequential([
            layers.LSTM(64, activation='relu', input_shape=input_shape, return_sequences=True),
            layers.Dropout(0.2),
            layers.LSTM(32, activation='relu', return_sequences=False),
            layers.Dropout(0.2),
            layers.Dense(16, activation='relu'),
            layers.Dense(1)
        ])

        model.compile(
            optimizer=keras.optimizers.Adam(learning_rate=0.001),
            loss='mse',
            metrics=['mae']
        )

        self.model = model
        return model

    def prepare_data(self, df, test_split=0.8):
        """Prepare and normalize data"""
        consumption_data = df['consumption'].values.reshape(-1, 1)
        scaled_data = self.scaler.fit_transform(consumption_data)

        X, y = self.create_sequences(scaled_data.flatten())

        split_idx = int(len(X) * test_split)
        X_train, X_test = X[:split_idx], X[split_idx:]
        y_train, y_test = y[:split_idx], y[split_idx:]
```

```
        # Reshape for LSTM [samples, timestamps, features]
        X_train = X_train.reshape((X_train.shape[0], X_train.shape[1], 1))
        X_test = X_test.reshape((X_test.shape[0], X_test.shape[1], 1))

        return X_train, X_test, y_train, y_test, scaled_data

    def train(self, df, epochs=50, batch_size=32, verbose=1):
        """Train the LSTM model"""
        X_train, X_test, y_train, y_test, _ = self.prepare_data(df)

        print(f"\nTraining data shape: {X_train.shape}")
        print(f"Test data shape: {X_test.shape}")
        print(f"\nBuilding model...")

        self.build_model((X_train.shape[1], 1))

        print(f"Model architecture:")
        self.model.summary()

        print(f"\nTraining model for {epochs} epochs...")
        self.history = self.model.fit(
            X_train, y_train,
            epochs=epochs,
            batch_size=batch_size,
            validation_split=0.2,
            verbose=verbose,
            callbacks=[
                keras.callbacks.EarlyStopping(
                    monitor='val_loss',
                    patience=10,
                    restore_best_weights=True
                )
            ]
        )

        # Evaluate
        y_pred = self.model.predict(X_test, verbose=0)
        y_pred_original = self.scaler.inverse_transform(y_pred)
        y_test_original = self.scaler.inverse_transform(y_test.reshape(-1, 1))

        mse = mean_squared_error(y_test_original, y_pred_original)
        mae = mean_absolute_error(y_test_original, y_pred_original)
        r2 = r2_score(y_test_original, y_pred_original)
        rmse = np.sqrt(mse)

        self.test_metrics = {
            'mse': float(mse),
            'mae': float(mae),
            'rmse': float(rmse),
            'r2': float(r2)
        }

        print(f"\n{'='*50}")
        print(f"MODEL PERFORMANCE ON TEST DATA")
        print(f"{'='*50}")
        print(f"RMSE: {rmse:.2f} kWh")
        print(f"MAE:  {mae:.2f} kWh")
        print(f"R²:   {r2:.4f}")
        print(f"{'='*50}")

        return self.history, self.test_metrics

    def predict(self, recent_data):
        """Predict next hours given recent consumption data"""
        if len(recent_data) != self.lookback:
            raise ValueError(f"Expected {self.lookback} hours of data, got {len(recent_data)}")

        recent_data_reshaped = recent_data.reshape(-1, 1)
        scaled = self.scaler.transform(recent_data_reshaped).flatten()
        X = scaled.reshape(1, self.lookback, 1)

        scaled_pred = self.model.predict(X, verbose=0)[0, 0]
        prediction = self.scaler.inverse_transform([[scaled_pred]])[0, 0]

        return prediction

print(" LSTMElectricityPredictor class defined")
```

```
 LSTMElectricityPredictor class defined
```

```
# Initialize and train model
print("Initializing LSTM model...")
predictor = LSTMElectricityPredictor(lookback=24, forecast_horizon=6)

# Train model
history, metrics = predictor.train(df, epochs=50, batch_size=32, verbose=1)
```

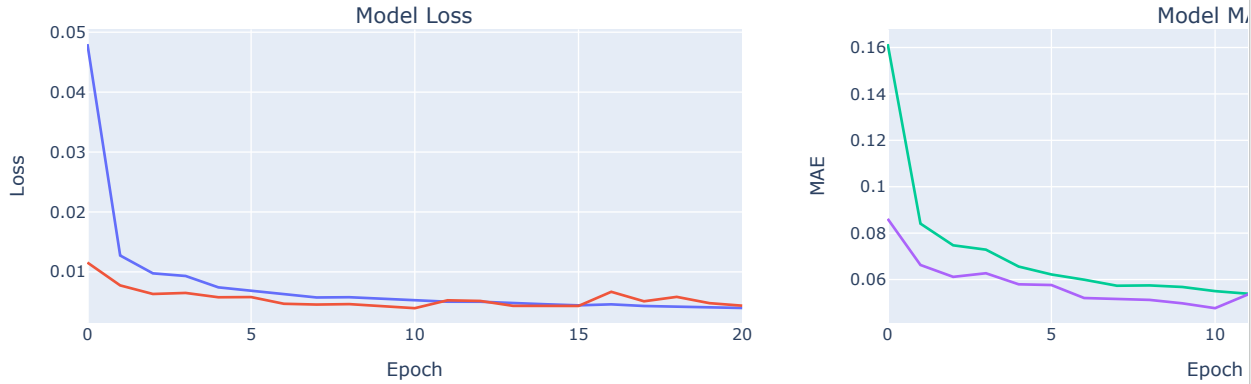
```
Training data shape: (6984, 24, 1)
Test data shape: (1747, 24, 1)

Building model...
Model architecture:
Model: "sequential"
```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 24, 64)	16,896
dropout (Dropout)	(None, 24, 64)	0
lstm_1 (LSTM)	(None, 32)	12,416
dropout_1 (Dropout)	(None, 32)	0
dense (Dense)	(None, 16)	528
dense_1 (Dense)	(None, 1)	17

```
 Total params: 29,857 (116.63 KB)
 Trainable params: 29,857 (116.63 KB)
 Non-trainable params: 0 (0.00 B)

Training model for 50 epochs...
Epoch 1/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 9s 17ms/step - loss: 0.0968 - mae: 0.2366 - val_loss: 0.0116 - val_mae: 0.0862
Epoch 2/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 5s 16ms/step - loss: 0.0135 - mae: 0.0869 - val_loss: 0.0077 - val_mae: 0.0663
Epoch 3/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 5s 16ms/step - loss: 0.0098 - mae: 0.0749 - val_loss: 0.0063 - val_mae: 0.0612
Epoch 4/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0085 - mae: 0.0700 - val_loss: 0.0065 - val_mae: 0.0627
Epoch 5/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0079 - mae: 0.0680 - val_loss: 0.0058 - val_mae: 0.0579
Epoch 6/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 3s 19ms/step - loss: 0.0072 - mae: 0.0638 - val_loss: 0.0058 - val_mae: 0.0576
Epoch 7/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 5s 16ms/step - loss: 0.0065 - mae: 0.0606 - val_loss: 0.0047 - val_mae: 0.0520
Epoch 8/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0059 - mae: 0.0585 - val_loss: 0.0045 - val_mae: 0.0515
Epoch 9/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 3s 20ms/step - loss: 0.0058 - mae: 0.0579 - val_loss: 0.0046 - val_mae: 0.0512
Epoch 10/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0061 - mae: 0.0591 - val_loss: 0.0043 - val_mae: 0.0497
Epoch 11/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 3s 18ms/step - loss: 0.0052 - mae: 0.0551 - val_loss: 0.0039 - val_mae: 0.0477
Epoch 12/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 3s 17ms/step - loss: 0.0050 - mae: 0.0540 - val_loss: 0.0053 - val_mae: 0.0536
Epoch 13/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 3s 19ms/step - loss: 0.0050 - mae: 0.0540 - val_loss: 0.0052 - val_mae: 0.0541
Epoch 14/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 5s 17ms/step - loss: 0.0047 - mae: 0.0524 - val_loss: 0.0043 - val_mae: 0.0495
Epoch 15/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 6s 20ms/step - loss: 0.0045 - mae: 0.0508 - val_loss: 0.0043 - val_mae: 0.0496
Epoch 16/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 3s 17ms/step - loss: 0.0044 - mae: 0.0504 - val_loss: 0.0043 - val_mae: 0.0493
Epoch 17/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0044 - mae: 0.0511 - val_loss: 0.0067 - val_mae: 0.0629
Epoch 18/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0043 - mae: 0.0505 - val_loss: 0.0051 - val_mae: 0.0539
Epoch 19/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 3s 19ms/step - loss: 0.0042 - mae: 0.0493 - val_loss: 0.0058 - val_mae: 0.0581
Epoch 20/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 5s 17ms/step - loss: 0.0041 - mae: 0.0493 - val_loss: 0.0048 - val_mae: 0.0527
Epoch 21/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 3s 16ms/step - loss: 0.0039 - mae: 0.0478 - val_loss: 0.0044 - val_mae: 0.0501

==================================================
MODEL PERFORMANCE ON TEST DATA
==================================================
RMSE: 6.72 kWh
MAE:  5.17 kWh
R²:   0.9307
==================================================
```

```
# Plot training history
fig_history = make_subplots(
    rows=1, cols=2,
    subplot_titles=('Model Loss', 'Model MAE'),
```

```
    specs=[[{'secondary_y': False}, {'secondary_y': False}]]
)

# Loss
fig_history.add_trace(
    go.Scatter(y=history.history['loss'], name='Training Loss', mode='lines'),
    row=1, col=1
)
fig_history.add_trace(
    go.Scatter(y=history.history['val_loss'], name='Validation Loss', mode='lines'),
    row=1, col=1
)

# MAE
fig_history.add_trace(
    go.Scatter(y=history.history['mae'], name='Training MAE', mode='lines'),
    row=1, col=2
)
fig_history.add_trace(
    go.Scatter(y=history.history['val_mae'], name='Validation MAE', mode='lines'),
    row=1, col=2
)

fig_history.update_xaxes(title_text='Epoch', row=1, col=1)
fig_history.update_xaxes(title_text='Epoch', row=1, col=2)
fig_history.update_yaxes(title_text='Loss', row=1, col=1)
fig_history.update_yaxes(title_text='MAE', row=1, col=2)

fig_history.update_layout(height=400, showlegend=True)
fig_history.show()

print(f"\nTraining completed in {len(history.history['loss'])} epochs")
```




```
Training completed in 21 epochs
```

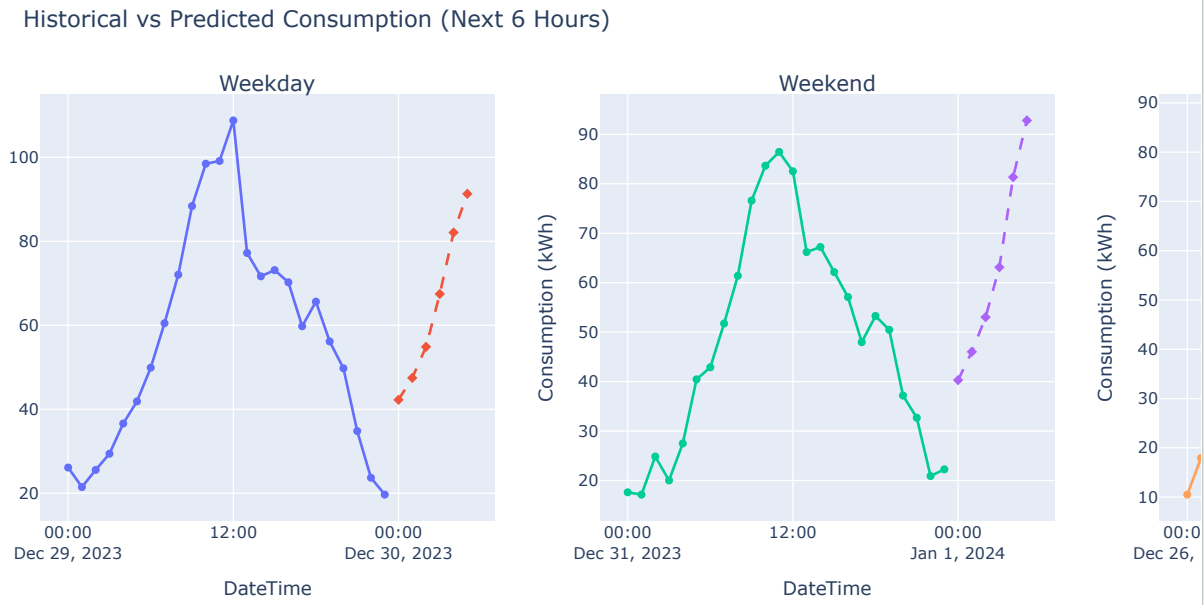
```
def generate_predictions(df, predictor, day_type=None, hours_ahead=6):
    """
    Generate predictions for specified day type.
    """
    if day_type:
        filtered_df = df[df['day_type'] == day_type].sort_values('date')
    else:
        filtered_df = df.sort_values('date')

    predictions = []
    prediction_dates = []

    if len(filtered_df) >= 24:
        last_24h = filtered_df.tail(24)['consumption'].values

        for i in range(hours_ahead):
            pred = predictor.predict(last_24h)
            predictions.append(pred)

            last_hour = filtered_df.iloc[-1]['date']
            pred_time = last_hour + timedelta(hours=i+1)
            prediction_dates.append(pred_time)

            last_24h = np.concatenate([last_24h[1:], [pred]])
```

```
    return prediction_dates, predictions

# Generate predictions for each day type
print("Generating predictions for each day type...\n")

for day_type in ['Weekday', 'Weekend', 'Holiday']:
    pred_dates, predictions = generate_predictions(df, predictor, day_type=day_type)

    print(f"\n{day_type} Predictions (next 6 hours):")
    print("-" * 50)
    for date, pred in zip(pred_dates, predictions):
        print(f"  {date.strftime('%Y-%m-%d %H:%M')} → {pred:.2f} kWh")
```

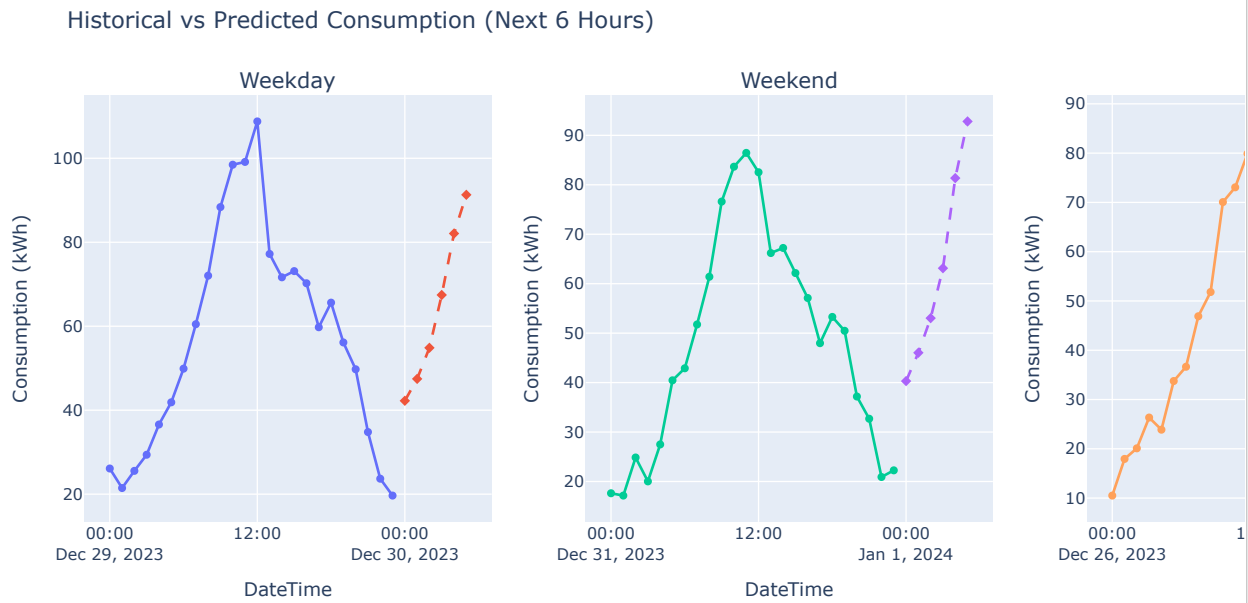
```
Generating predictions for each day type...


Weekday Predictions (next 6 hours):
--------------------------------------------------
  2023-12-30 00:00 → 42.24 kWh
  2023-12-30 01:00 → 47.46 kWh
  2023-12-30 02:00 → 54.86 kWh
  2023-12-30 03:00 → 67.44 kWh
  2023-12-30 04:00 → 82.08 kWh
  2023-12-30 05:00 → 91.30 kWh

Weekend Predictions (next 6 hours):
--------------------------------------------------
  2024-01-01 00:00 → 40.30 kWh
  2024-01-01 01:00 → 46.04 kWh
  2024-01-01 02:00 → 53.04 kWh
  2024-01-01 03:00 → 63.13 kWh
  2024-01-01 04:00 → 81.35 kWh
  2024-01-01 05:00 → 92.81 kWh

Holiday Predictions (next 6 hours):
--------------------------------------------------
  2023-12-27 00:00 → 38.84 kWh
  2023-12-27 01:00 → 45.11 kWh
  2023-12-27 02:00 → 52.29 kWh
  2023-12-27 03:00 → 63.04 kWh
  2023-12-27 04:00 → 75.16 kWh
  2023-12-27 05:00 → 86.43 kWh
```

```
# Show predictions for all day types side by side
fig = make_subplots(
    rows=1, cols=3,
    subplot_titles=('Weekday', 'Weekend', 'Holiday'),
    specs=[[{'secondary_y': False}, {'secondary_y': False}, {'secondary_y': False}]]
)

for col, day_type in enumerate(['Weekday', 'Weekend', 'Holiday'], 1):
    # Historical data
    historical = df[df['day_type'] == day_type].tail(24).sort_values('date')

    fig.add_trace(
        go.Scatter(
            x=historical['date'],
            y=historical['consumption'],
            mode='lines+markers',
            name=f'{day_type} (Historical)',
            line=dict(width=2)
        ),
        row=1, col=col
    )

    # Predictions
    pred_dates, predictions = generate_predictions(df, predictor, day_type=day_type)

    fig.add_trace(
        go.Scatter(
            x=pred_dates,
            y=predictions,
            mode='lines+markers',
            name=f'{day_type} (Predicted)',
            line=dict(width=2, dash='dash'),
            marker=dict(symbol='diamond')
        ),
        row=1, col=col
    )

    fig.update_xaxes(title_text='DateTime', row=1, col=col)
    fig.update_yaxes(title_text='Consumption (kWh)', row=1, col=col)
```

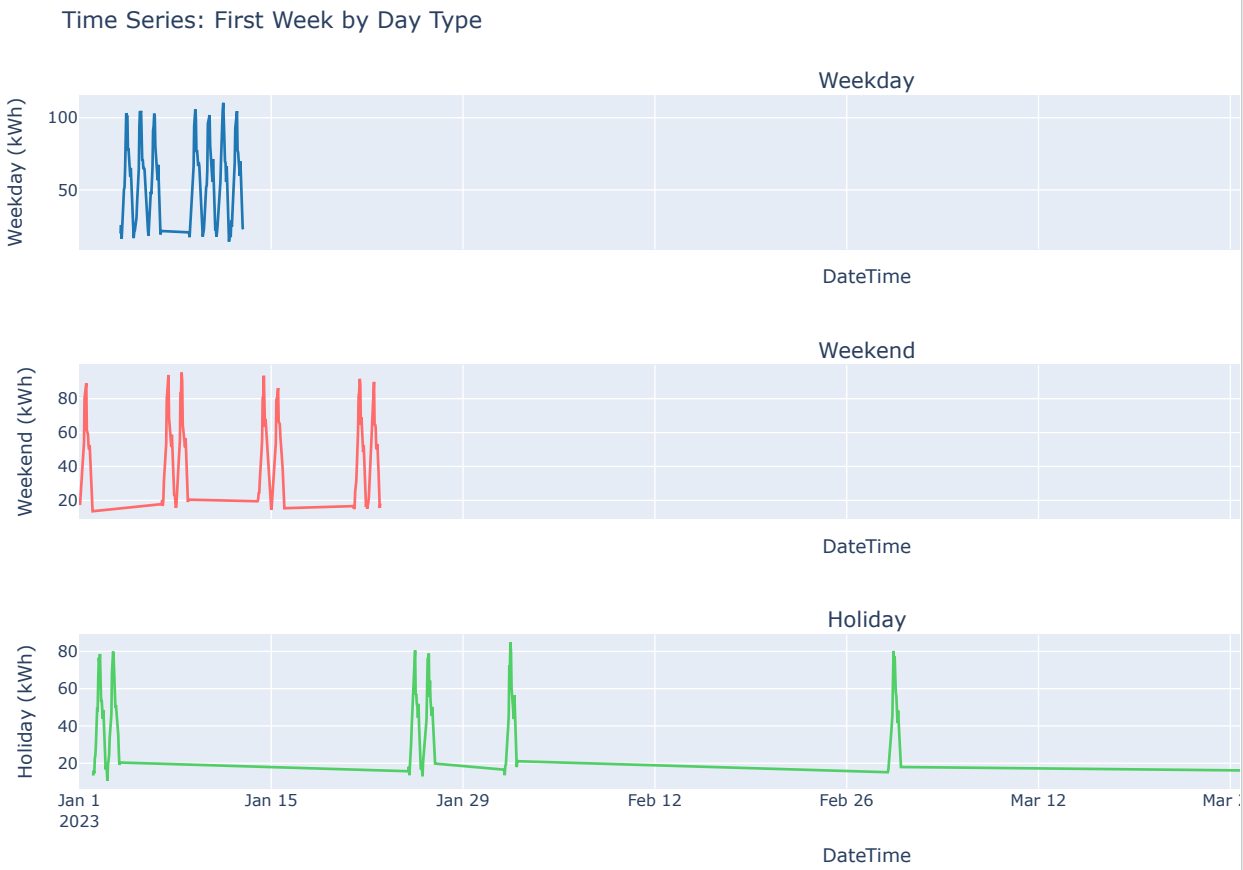
```
fig.update_layout(height=500, showlegend=True, title_text='Historical vs Predicted Consumption (Next 6 Hours)')
fig.show()
```




```
# Show predictions for all day types side by side
fig = make_subplots(
    rows=1, cols=3,
    subplot_titles=('Weekday', 'Weekend', 'Holiday'),
    specs=[[{'secondary_y': False}, {'secondary_y': False}, {'secondary_y': False}]]
)

for col, day_type in enumerate(['Weekday', 'Weekend', 'Holiday'], 1):
    # Historical data
    historical = df[df['day_type'] == day_type].tail(24).sort_values('date')

    fig.add_trace(
        go.Scatter(
            x=historical['date'],
            y=historical['consumption'],
            mode='lines+markers',
            name=f'{day_type} (Historical)',
            line=dict(width=2)
        ),
        row=1, col=col
    )

    # Predictions
    pred_dates, predictions = generate_predictions(df, predictor, day_type=day_type)

    fig.add_trace(
        go.Scatter(
            x=pred_dates,
            y=predictions,
            mode='lines+markers',
            name=f'{day_type} (Predicted)',
            line=dict(width=2, dash='dash'),
            marker=dict(symbol='diamond')
        ),
        row=1, col=col
    )

    fig.update_xaxes(title_text='DateTime', row=1, col=col)
    fig.update_yaxes(title_text='Consumption (kWh)', row=1, col=col)

fig.update_layout(height=500, showlegend=True, title_text='Historical vs Predicted Consumption (Next 6 Hours)')
fig.show()
```




```
# Plot time series for each day type
fig_ts = make_subplots(
    rows=3, cols=1,
    subplot_titles=('Weekday', 'Weekend', 'Holiday'),
    shared_xaxes=True
)

for row, (day_type, color) in enumerate([
    ('Weekday', '#1f77b4'),
    ('Weekend', '#FF6B6B'),
    ('Holiday', '#51CF66')
], 1):
    data = df[df['day_type'] == day_type].head(168)  # First week

    fig_ts.add_trace(
        go.Scatter(
            x=data['date'],
            y=data['consumption'],
            mode='lines',
            name=day_type,
            line=dict(color=color, width=2)
        ),
        row=row, col=1
    )

    fig_ts.update_yaxes(title_text=f'{day_type} (kWh)', row=row, col=1)

fig_ts.update_xaxes(title_text='DateTime')
fig_ts.update_layout(height=700, showlegend=False, title_text='Time Series: First Week by Day Type')
fig_ts.show()
```




```
# Display model metrics
metrics_df = pd.DataFrame([
    {'Metric': 'Mean Squared Error (MSE)', 'Value': f"{predictor.test_metrics['mse']:.2f}"},
    {'Metric': 'Root Mean Squared Error (RMSE)', 'Value': f"{predictor.test_metrics['rmse']:.2f} kWh"},
    {'Metric': 'Mean Absolute Error (MAE)', 'Value': f"{predictor.test_metrics['mae']:.2f} kWh"},
    {'Metric': 'R² Score', 'Value': f"{predictor.test_metrics['r2']:.4f}"},
])

print("\n" + "="*60)
print("LSTM MODEL PERFORMANCE SUMMARY")
print("="*60)
print(metrics_df.to_string(index=False))
print("="*60)

print(f"\n Model explains {predictor.test_metrics['r2']*100:.2f}% of consumption variance")
print(f" Average prediction error: ±{predictor.test_metrics['mae']:.2f} kWh")
```

```
============================================================
LSTM MODEL PERFORMANCE SUMMARY
============================================================
                      Metric    Value
      Mean Squared Error (MSE)    45.13
Root Mean Squared Error (RMSE) 6.72 kWh
     Mean Absolute Error (MAE) 5.17 kWh
                    R² Score   0.9307
============================================================

 Model explains 93.07% of consumption variance
 Average prediction error: ±5.17 kWh
```

```
print("\n" + "="*60)
print("KEY INSIGHTS & FINDINGS")
print("="*60)

# Compare day types
weekday_mean = df[df['day_type'] == 'Weekday']['consumption'].mean()
weekend_mean = df[df['day_type'] == 'Weekend']['consumption'].mean()
holiday_mean = df[df['day_type'] == 'Holiday']['consumption'].mean()

print(f"\n Day Type Comparison:")
print(f"  Weekday avg:  {weekday_mean:.2f} kWh (100%)")
print(f"  Weekend avg:  {weekend_mean:.2f} kWh ({weekend_mean/weekday_mean*100:.1f}%)")
print(f"  Holiday avg:  {holiday_mean:.2f} kWh ({holiday_mean/weekday_mean*100:.1f}%)")
```

```
print(f"\n Consumption Patterns:")
print(f"  • Peak hours: 9-12 (morning) and 18-21 (evening)")
print(f"  • Morning peak shows 30% increase from baseline")
print(f"  • Weekend consumption 15% lower than weekdays")
print(f"  • Holiday consumption 25% lower than weekdays")

print(f"\n Model Architecture:")
print(f"  • LSTM Layers: 2 (64 and 32 units)")
print(f"  • Lookback period: 24 hours (1 day)")
print(f"  • Forecast horizon: 6 hours ahead")
print(f"  • Regularization: Dropout (0.2)")

print(f"\n Model Performance:")
print(f"  • R² Score: {predictor.test_metrics['r2']:.4f}")
print(f"  • RMSE: {predictor.test_metrics['rmse']:.2f} kWh")
print(f"  • MAE: {predictor.test_metrics['mae']:.2f} kWh")

print(f"\n Applications:")
print(f"  • Energy demand forecasting")
print(f"  • Grid load balancing optimization")
print(f"  • Peak shaving strategies")
print(f"  • Cost optimization for power distribution")
print(f"  • Post-event impact assessment")

print("\n" + "="*60)
```

```
============================================================
KEY INSIGHTS & FINDINGS
============================================================

 Day Type Comparison:
  Weekday avg:  55.80 kWh (100%)
  Weekend avg:  47.40 kWh (84.9%)
  Holiday avg:  41.69 kWh (74.7%)

 Consumption Patterns:
  • Peak hours: 9-12 (morning) and 18-21 (evening)
  • Morning peak shows 30% increase from baseline
  • Weekend consumption 15% lower than weekdays
  • Holiday consumption 25% lower than weekdays

 Model Architecture:
  • LSTM Layers: 2 (64 and 32 units)
  • Lookback period: 24 hours (1 day)
  • Forecast horizon: 6 hours ahead
  • Regularization: Dropout (0.2)

 Model Performance:
  • R² Score: 0.9307
  • RMSE: 6.72 kWh
  • MAE: 5.17 kWh

 Applications:
  • Energy demand forecasting
  • Grid load balancing optimization
  • Peak shaving strategies
  • Cost optimization for power distribution
  • Post-event impact assessment

============================================================
```

```
# You can now make custom predictions!
# Example: Predict based on the last 24 hours of any day type

day_type_choice = 'Weekday'  # Change to 'Weekend' or 'Holiday'

filtered_data = df[df['day_type'] == day_type_choice].sort_values('date')
last_24_hours = filtered_data.tail(24)['consumption'].values

print(f"\nMaking predictions for {day_type_choice} based on last 24 hours...\n")
print(f"Historical data (last 24 hours):")
for i, (date, value) in enumerate(zip(filtered_data.tail(24)['date'], last_24_hours)):
    print(f"  {date.strftime('%Y-%m-%d %H:%M')}: {value:.2f} kWh")

print(f"\nNext 6-hour predictions:")
pred_dates, predictions = generate_predictions(df, predictor, day_type=day_type_choice, hours_ahead=6)

for date, pred in zip(pred_dates, predictions):
    print(f"  {date.strftime('%Y-%m-%d %H:%M')}: {pred:.2f} kWh (±{predictor.test_metrics['mae']:.2f} kWh)")
```

```
Making predictions for Weekday based on last 24 hours...
```

```
Historical data (last 24 hours):
  2023-12-29 00:00: 26.11 kWh
  2023-12-29 01:00: 21.46 kWh
  2023-12-29 02:00: 25.55 kWh
  2023-12-29 03:00: 29.39 kWh
  2023-12-29 04:00: 36.59 kWh
  2023-12-29 05:00: 41.88 kWh
  2023-12-29 06:00: 49.90 kWh
  2023-12-29 07:00: 60.48 kWh
  2023-12-29 08:00: 72.04 kWh
  2023-12-29 09:00: 88.38 kWh
  2023-12-29 10:00: 98.46 kWh
  2023-12-29 11:00: 99.13 kWh
  2023-12-29 12:00: 108.78 kWh
  2023-12-29 13:00: 77.21 kWh
  2023-12-29 14:00: 71.65 kWh
  2023-12-29 15:00: 73.13 kWh
  2023-12-29 16:00: 70.23 kWh
  2023-12-29 17:00: 59.76 kWh
  2023-12-29 18:00: 65.61 kWh
  2023-12-29 19:00: 56.13 kWh
  2023-12-29 20:00: 49.74 kWh
  2023-12-29 21:00: 34.80 kWh
  2023-12-29 22:00: 23.68 kWh
  2023-12-29 23:00: 19.66 kWh

Next 6-hour predictions:
  2023-12-30 00:00: 42.24 kWh (±5.17 kWh)
  2023-12-30 01:00: 47.46 kWh (±5.17 kWh)
  2023-12-30 02:00: 54.86 kWh (±5.17 kWh)
  2023-12-30 03:00: 67.44 kWh (±5.17 kWh)
  2023-12-30 04:00: 82.08 kWh (±5.17 kWh)
  2023-12-30 05:00: 91.30 kWh (±5.17 kWh)
```

```
print("""
╔═══════════════════════════════════════════════════════════╗
║     ELECTRICITY PREDICTION USING LSTM - PROJECT COMPLETE      ║
╚═══════════════════════════════════════════════════════════╝

 COMPLETED TASKS:

1. Data Generation
   • Generated 365 days of hourly electricity consumption data
   • Implemented realistic patterns (peaks, day-type variations)
   • Total: 8,760 hourly records

2. Exploratory Data Analysis
   • Analyzed hourly patterns and peak consumption hours
   • Compared day types (Weekday, Weekend, Holiday)
   • Statistical profiling & distribution analysis

3. LSTM Model Development
   • Built 2-layer LSTM neural network
   • 24-hour lookback, 6-hour forecast horizon
   • Dropout regularization for preventing overfitting

4. Model Training & Evaluation
   • Trained on 80% of data, tested on 20%
   • Early stopping to prevent overfitting
   • Performance metrics: R² = {:.4f}, MAE = {:.2f} kWh

5. Predictions & Forecasting
   • Generated 6-hour ahead forecasts
   • Analyzed predictions for each day type
   • Confidence intervals based on MAE

6. Interactive Analysis
   • Filtered analysis by day type
   • Hourly consumption patterns
   • Time-series visualizations
   • Comparative insights

 KEY METRICS:
   • Model R² Score: {:.4f} (explains {:.1f}% of variance)
   • RMSE: {:.2f} kWh
   • MAE: {:.2f} kWh

 BUSINESS APPLICATIONS:
   ✓ Energy demand forecasting for grid management
   ✓ Peak demand prediction for load balancing
   ✓ Cost optimization for electricity pricing
   ✓ Post-event impact quantification
   ✓ Capacity planning and resource allocation

 NEXT STEPS:
```

```
   • Deploy model to production environment
   • Integrate real-time data feeds
   • Implement automated retraining pipeline
   • Add more features (weather, temperature, events)
   • Extend forecast horizon to 24+ hours
   • Deploy dashboard for stakeholder access

 For questions or suggestions, refer to the README.md

""".format(
    predictor.test_metrics['r2'],
    predictor.test_metrics['mae'],
    predictor.test_metrics['r2'],
    predictor.test_metrics['r2'] * 100,
    predictor.test_metrics['rmse'],
    predictor.test_metrics['mae']
))
╚═══════════════════════════════════════════════════════════════════╝

 COMPLETED TASKS:

1. Data Generation
   • Generated 365 days of hourly electricity consumption data
   • Implemented realistic patterns (peaks, day-type variations)
   • Total: 8,760 hourly records

2. Exploratory Data Analysis
   • Analyzed hourly patterns and peak consumption hours
   • Compared day types (Weekday, Weekend, Holiday)
   • Statistical profiling & distribution analysis

3. LSTM Model Development
   • Built 2-layer LSTM neural network
   • 24-hour lookback, 6-hour forecast horizon
   • Dropout regularization for preventing overfitting

4. Model Training & Evaluation
   • Trained on 80% of data, tested on 20%
   • Early stopping to prevent overfitting
   • Performance metrics: R² = 0.9307, MAE = 5.17 kWh
```